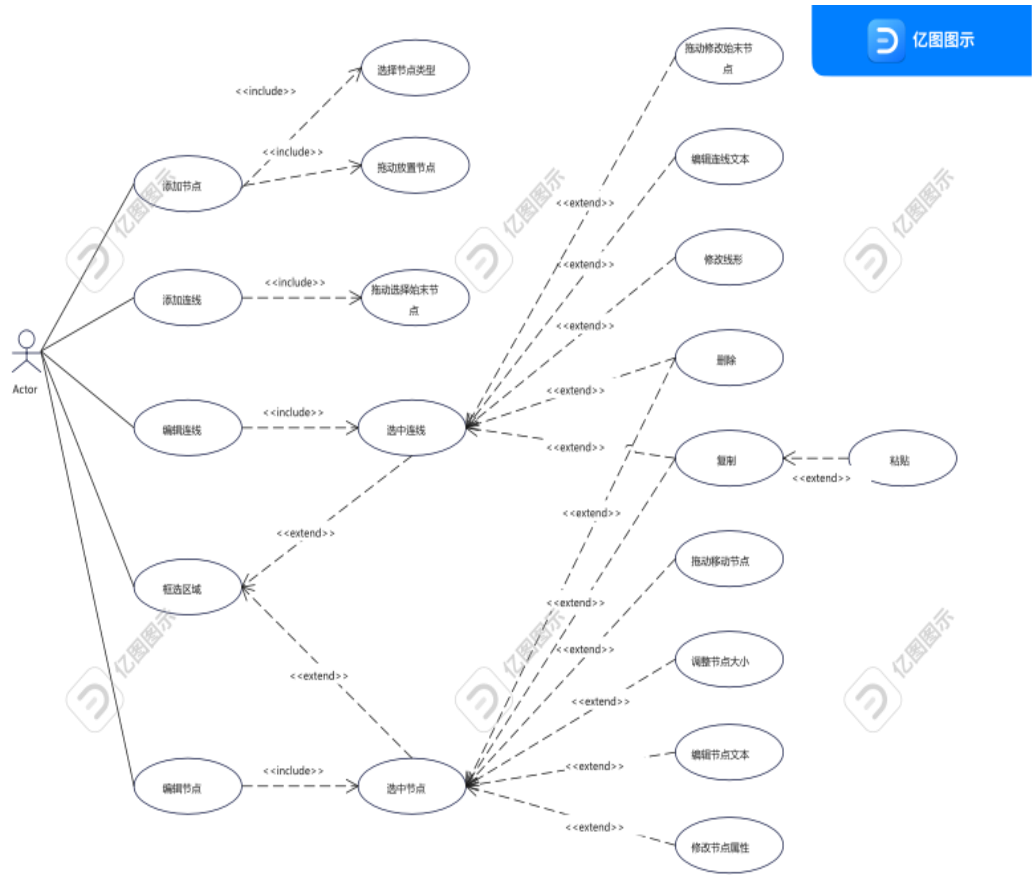


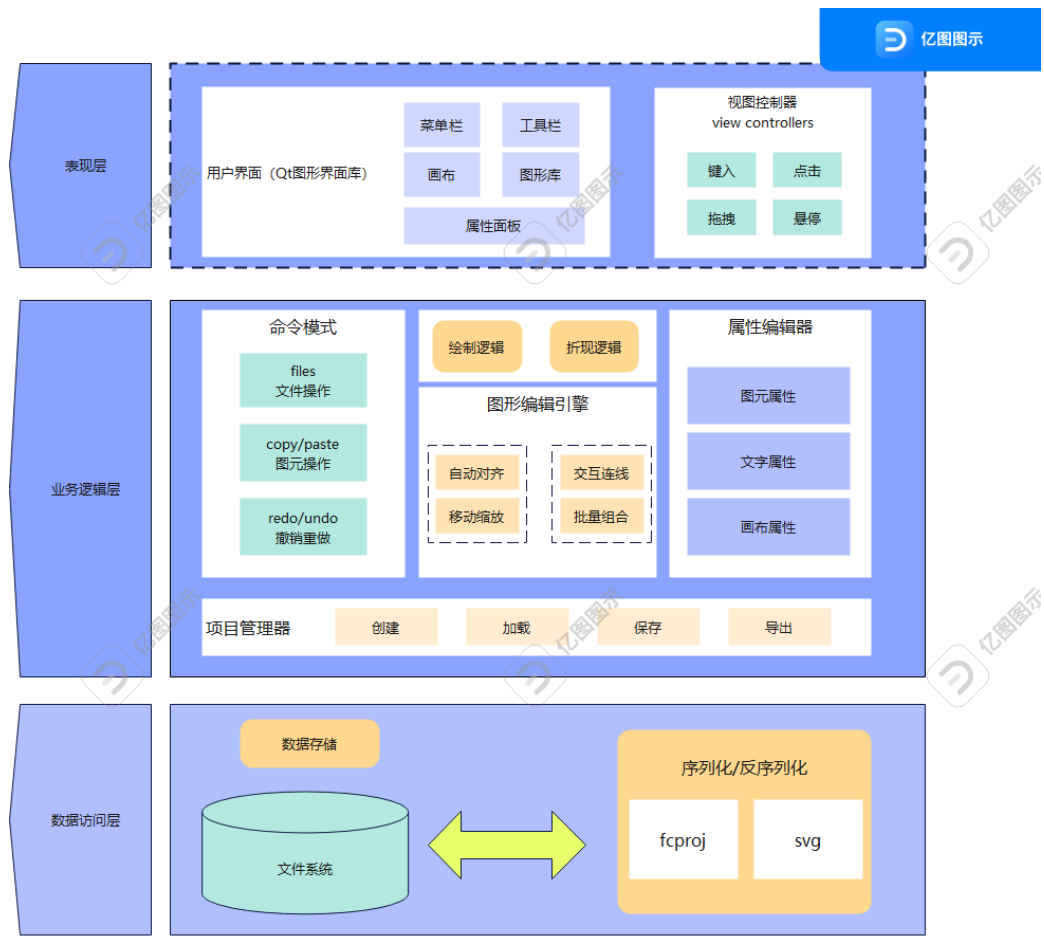
流程图编辑器设计文档

一、整体架构设计

1、用例图：



2、架构图：



二、 数据结构设计

1、DiagramItem

父类：	QGraphicsItem	
关键成员	类型	说明
DiagramType	enum	图元的 20 种类型
TransformState	enum	记录八个连接点的位置
m_grapSize	QSize	记录图元的尺寸

m_minSize	QSize	记录图元的最小尺寸
m_tfState	DiagramType	本图元实例的类型
textItem	QGraphicsTextItem*	图元的文本框
pathes	QList<DiagramPath *>	记录连接到本图元的所有路径
marks	QMap<DiagramPath*,QString>	记录连接路径的连接点和起终点信息
关键方法	类型	说明
paint	void	图元的绘制方法，根据类型选择不同的绘画方法，同时绘制八个缩放点和四个连接点
hoverMoveEvent	void	重写悬停方法，设置悬停鼠标样式
mouseMoveEvent	void	图元移动和缩放

2、DiagramPath

父类	QGraphicsPathItem	
关键成员	类型	说明

startItem	DiagramItem*	路径的起始图元
endItem	DiagramItem*	路径的终止图元
startState	DiagramItem::TransformState	起始图元的连接 点
endState	DiagramItem::TransformState	终止图元的连接 点
m_path	QPainterPath	路径信息
m_quad	int	起点终点的相对 位置，象限
m_state	int	根据起始终止图 元的连接点信息 和相对位置信息 确定的状态参数
关键方法	类型	说明
updatePath	void	绘制路径/更新路 径
drawHead	void	路径末尾绘制箭 头
quad	int	计算象限
drawZig	void	实现折线算法绘 制折线

3、DiagramItemGroup

父类	QGraphicsItemGroup	
关键成员	类型	说明
top	qreal	计算组合的最大顶边界
bottom	qreal	计算组合的最小底边界
leftBound	qreal	计算组合的最大左边界
rightBound	qreal	计算组合的最小右边界
m_grapSize	QSize	组合框的尺寸
m_minSize	QSize	组合框的最小尺寸
m_border	int	组合框缩放点的半边长
关键方法	类型	说明
paint	void	组合的绘制方法，调用每个子类的 paint 方法，同时绘制八个缩放点和四个连接点
hoverMoveEvent	void	重写悬停方法，设置悬停鼠标样式
mouseMoveEvent	void	组合移动和缩放，子类按比例进行缩放

4、DiagramScene

父类	QGraphicsScene
----	----------------

关键成员	类型	说明
Mode	enum	包含{InsertItem, InsertLine, InsertText, MoveItem, InsertPath}五个取值, 表示当前 scene 正在进行的操作
menu;	QMenu *	画布右键菜单
myItemMenu	QMenu *	图元右键菜单
pathLine	QGraphicsLineItem*	从连接点拉出的示意线
myMode	Mode	当前操作状态
premode	Mode	上一个操作状态
alignedItem	QGraphicsItem *	当前对齐的图元
movedItem	QGraphicsItem *	当前正在拖动的图元
iscenterY	bool	标记是否进行中心对齐 (纵向)
iscenterX	bool	标记是否进行中心对齐 (横向)
isbottom	bool	标记是否进行底部对齐
isright	bool	标记是否进行右端对齐
istop	bool	标记是否进行顶端对齐
isleft	bool	标记是否进行左端对齐
alignPosition	QPointF	对齐基准点

rectItem	QGraphicsRectItem *	框选时出现的蓝色边框
ischeckboxingbox	bool	判断是否正在框选
endpoint	QPointF	鼠标框选末位置
beginpoint	QPointF	鼠标框选起始位置
myItemType	DiagramItem::DiagramType	即将插入的图元类型
leftButtonDown	bool	左键按下
关键方法	类型	说明
setLinkVisible(bool b)	void	设置所有图元磁力点可见性
setMode(Mode mode)	void	设置当前操作类型
createItem(int type);	QGraphicsItem*	创建图元
createTextItem();	QGraphicsTextItem*	创建文本
mouseReleaseEvent(QGraphicsSceneMouseEvent *mouseEvent) override;	void	重写的鼠标释放事件，根据当前模式执行操作
drawForeground(QPainter *painter,	void	执行绘制辅助线相关操作

const QRectF &rect) override;		
keyPressEvent(QKeyEvent *event) override;	void	重写的键盘按下事件， 根据当前模式执行操作
mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent) override;	void	重写的鼠标按下事件， 根据当前模式执行操作
mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent) override;	void	重写的鼠标移动事件， 根据当前模式执行操作

5、DiagramTextItem

父类	QGraphicsTextItem	
关键成员	类型	说明
text_color	QColor	文本颜色
关键方法	类型	说明

itemChange(GraphicsItemChange change, const QVariant &value)	QVariant	处理图形项状态的变化，发送信号 selectedChange
focusOutEvent(QFocusEvent *event)	void	处理焦点移出事件，发送 lostFocus 信号并取消文本编辑模式
mouseDoubleClickEvent(QGraphicsSceneMouseEvent *event)	void	双击鼠标时进入文本编辑模式
contextMenuEvent(QGraphicsSceneContextMenuEvent *event)	void	右键菜单事件，显示默认的上下文菜单

6、DeleteCommand

父类	QUndoCommand	
关键成员	类型	说明
m_itemPos	QPointF	记录当前 item 位置
m_itemSize	QSizeF	记录当前 item 大小
关键方法	类型	说明
undo()	void	删除撤销操作
redo()	void	删除重做操作

7、FindReplaceDialog

父类	QDialog	
关键成员	类型	说明
findLineEdit	QLineEdit*	输入查找文本的编辑框
replaceLineEdit	QLineEdit*	输入替换文本的编辑框
findButton	QPushButton*	“查找下一个”按钮
replaceButton	QPushButton*	“替换”按钮
replaceAllButton	QPushButton*	“全部替换”按钮

关键方法	类型	说明
onFindClicked()	void	处理“查找下一个”按钮点击事件, 发送 findText 信号
onReplaceClicked()	void	处理“替换”按钮点击事件, 发送 replaceText 信号
onReplaceAllClicked()	void	处理“全部替换”按钮点击事件, 发送 replaceAllText 信号

8、MainWindow

父类	QMainWindow	
关键成员	类型	说明
scene	DiagramScene *	当前场景
view	QGraphicsView *	当前视图（画布上显示的内容）
tabwidget	QTabWidget *	QT 标签页部件, 用于实现多开和切换
globalTabCounter	int	全局变量, 用于统计当前 QT 标签页数
sceneVector	QVector<DiagramScene*>	用于存储同时打开的 DiagramScene
viewVector	QVector<QGraphicsView*>	用于存储同时打开的 DiagramView
undoStack	QStack<QString>	undo 栈, 储存一系列文件的地址

redoStack	QStack<QString>	redo 栈，功能同上
fileMenu	QMenu *	文件下拉菜单
itemMenu	QMenu *	编辑下拉菜单
aboutMenu	QMenu *	帮助下拉菜单
savePicPath	QString	记录上次图形存储地址
saveFilePath	QString	记录上次文件存储地址
saveFileSuccess	bool	判读文件保存是否成功
icon	QIcon	程序运行时显示的图标
各种 Action	QAction	作为 UI 栏中可以随时插入使用的功能组件
WriteDiagramItem	struct	结构体负责存储 scene 中图形的属性参数
ReadDiagramItem	struct	结构体负责存储 fcproj 自定义工程文件中图形的属性参数
WriteDiagramPath	struct	结构体负责存储 scene 中连线的属性参数
ReadDiagramPath	struct	结构体负责存储 fcproj 自定义工程文件中连线的属性参数
关键方法	类型	说明

newScene()	void	执行创建新画布的相关操作（包括设置尺寸、将场景和视图分别添加至容器、将 tab 添加至 tabwidget、连接信号与槽等），并且将得到的 Scene 的视图中心设置为 0,0 左上角对齐，并且更新 globalTab Counter 从而实现新标签页面的标题更新，最终实现较好的新增页面效果
sceneChanged()	void	槽函数，与 tabwidget 页面切换信号相连，负责重新连接槽函数并更新当前页面状态

closeScene(int index)	void	槽函数，与 tabwidget 页面关闭请求信号相 连，负责判断禁止移除 最后一个页面，否则从 容器中移除场景和视 图，从 tabwidget 中移除 tab
createCellWidget(co nst QString &text, DiagramItem::Diagra mType type)	QWidget	设置左侧图源集中对应 图源显示的长度宽度等 等，从而控制左侧选择 栏中 button 的大小和样 式
saveSceneAsImageO rSvg()	bool	将 Scene 中的所有图像 元素利用 pixmap 获取并 存储为 jpeg/png 类型文 件；或者利用 QSvgGenerator 将当前 Scene 直接转化为 Svg 文 件

changebackground()	void	允许在电脑磁盘中选择 *.png *.jpg *.bmp 类型的文件图像，利用 QPixmap 将其转换成一个可以复制铺满的有效遮罩，从而将其设为当前 tab 的背景
saveSaveFilePath(const QString &filePath)	void	通过将工程文件存储地址作为 QString 存储到当前目录下的特定文件中从而实现存储路径保存功能 防止因程序关闭而导致存储记忆消失
loadSaveFilePath()	QString	通过将当前目录下的特定文件中读取工程文件存储地址作为 QString 从而实现存储路径读取功能 防止因程序关闭而导致存储记忆消失

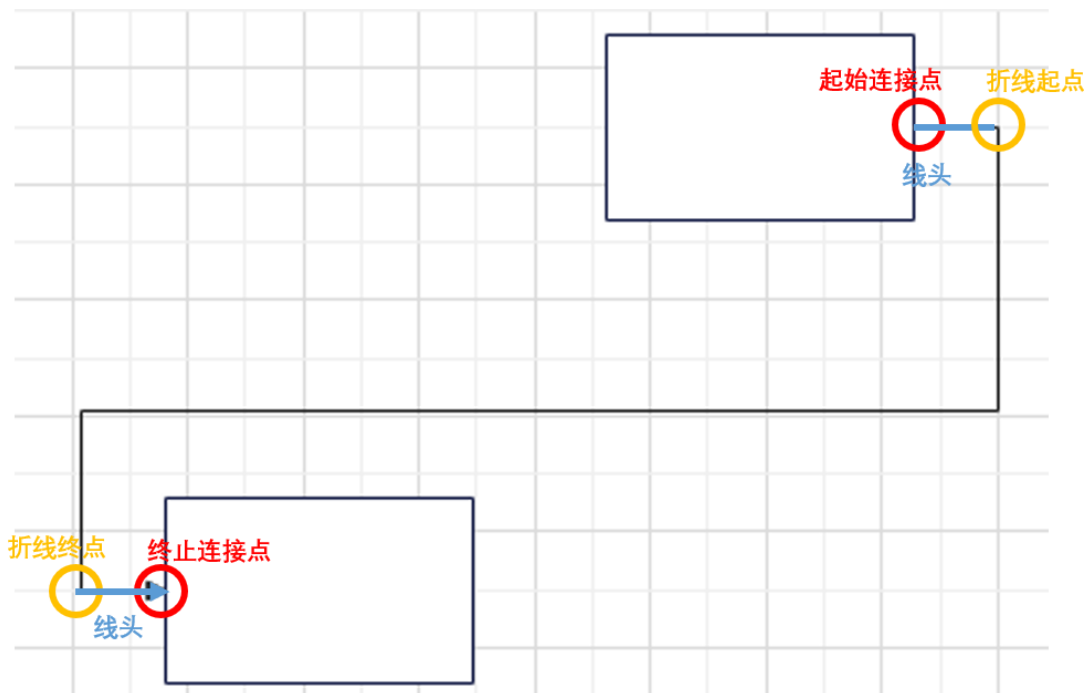
savefile()	void	利用 QString 将得到的 WriteDiagramItem 和 WriteDiagramPath 结构体列表序列化为二进制数据存储在特定的文件类型 fcproj 之中，并且将 QString 中自带的空格统一替换为*从而避免出现读取问题
loadfile()	void	利用 QString 将序列化存储二进制数据的文件类型 fcproj 打开，并且将其中的内容进行反序列化操作，利用空格进行数据的区分，并且在之后将其字符串中的*替换回空格，最后将得到的结果赋值给 ReadDiagramItem 和 ReadDiagramPath 列表从而将 fcproj 中的数据读取到 QT 程序中

createActions	void	创建各种 Action ，并且为其赋予相应的文字说明，快捷键和图标以及信号槽，从而便于在例如 ToolBar 这样的工具栏创建过程中功能组件的添加
----------------------	-------------	---

三、 关键算法设计

1、折线算法

在进行折线算法前，程序已绘制好了线头，即根据连接点在图元上的位置，进行一段固定的延伸。自起始连接点向外延伸得到的点即为折线算法的起点，自终止连接点向外延伸得到的点即为折线算法的终点。



折线算法的输入是起点方向、终点方向、起终点相对位置，例如上图就是一个向右发出、向左进入、终点位于起点第三象限的情形，记为“ $\rightarrow, \leftarrow, 3$ ”。所有的输入共有 $4*4*4=64$ 种情形。

令起点坐标为 (x_1, y_1) ，终点坐标为 (x_2, y_2) 。在禁止折线与线头重合的情况下，从起点到终点的移动只有四种方式。其中一种为： $(x_1, y_1) \rightarrow (x_2, y_1) \rightarrow (x_2, y_2)$ ，将其记为“XY”，意为先改变 x 坐标再改变 y 坐标；另一种为： $(x_1, y_1) \rightarrow (x_1, (y_1+y_2)/2) \rightarrow ((x_1+x_2)/2, (y_1+y_2)/2) \rightarrow ((x_1+x_2)/2, y_2) \rightarrow (x_2, y_2)$ ，记为“YXY”，示意图中就是“YXY”的一个实例。以此类推，折线移动方式分为以下四种：XY、YX、XYX、YXY。

接下来需要 64 种输入归类到 4 种折线移动方式中。虽然该归类存在一定规律，但难以用代码表示，且表示规律的代价或许

高于穷举 64 种情况，我们选择人工进行归类。

完成归类后，我们根据之前设置的方向枚举对 64 种情形进行散列编码，进行条件判断执行相应的 4 种操作，完成了该算法的实现。YXY 情形的相关代码如下图所示。

```
case 841:
case 844:
case 123:
case 124:
case 482:
case 483:
case 211:
case 212:{
    m_path.lineTo(startPoint.x(),midPoint.y());
    m_path.lineTo(endPoint.x(),midPoint.y());
    break;
}
```

2、撤销重做算法

本项目的撤销重做功能在文件保存功能的基础上进行实现，因为需要对画布进行“快照”，保存当前画布上的一切图元信息。

其基本原理的实现需要一个 **undo** 栈和一个 **redo** 栈（本质上是两个保存了一些列项目文件的文件夹）。基本原理如下：

当发生需要记录的操作时（例如加入图元、删除图元），对当前画布进行保存，压入 **undo** 栈。

当执行 **undo** 操作时，弹出 **undo** 栈栈顶的画布状态，读取该状态到当前画布，并将该状态压入 **redo** 栈。

当执行 **redo** 操作时，弹出 **redo** 栈栈顶的画布状态并读取，同时将该状态压入 **undo** 栈。

该功能算法较为简单，其实现效果主要取决于对“关键操作”

的选择和获取。例如：若把鼠标选中加入关键操作将毫无意义；若图元任何位置改变都要触发保存，则保存的状态数会很快超过栈容量（因为每次人为拖动都伴随着多次位置变化）。